

LABORATORIO N° 06

Desarrollo de Software



DOCENTE:
Bestard Aroche, Yunion

CURSO:

Desarrollo de Aplicaciones
Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad del motor de plantillas de Django (herencia, filtros, inclusión) dentro de una aplicación empresarial.**
2. **Refactorizar Views y Templates ya existentes para que reutilicen una estructura común mediante herencia de plantillas (extends / block).**
3. **Aplicar la sintaxis avanzada de plantillas (filtros, comentarios, include) y verificar el auto-escape como mecanismo de seguridad.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 6 (motor de plantillas de Django: herencia, filtros, include, seguridad) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación con CRUD completo (URLs, Views y Templates) desarrollada desde la Semana 3, ampliada con relaciones en la Semana 4 y registrada en el Django Admin en la Semana 5.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (antes y después del cambio) en el Template correspondiente, además de la página renderizada en el navegador.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Desde la Semana 3, cada estudiante ya cuenta con URLs, Views y Templates propios que implementan CRUD completo, ampliados en la Semana 4 con relaciones y expuestos también en el Django Admin en la Semana 5. En esta sesión no se crean Views ni Templates nuevos desde cero: se refactorizan los que ya existen para que reutilicen una estructura común y apliquen buenas prácticas de plantillas.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de la Semana 4/5 y migra sus Templates existentes a herencia con `base.html`, agrega filtros, comentarios e incluye. En la segunda parte, deberá hacer lo mismo sobre los Templates de su investigación propia (las siete entidades ya registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que la sintaxis de plantillas no reemplaza el CRUD que ya construyó: lo hace más mantenible y seguro. El mismo Template que ya muestra sus datos pasa a heredar una estructura común, a reutilizar fragmentos con `include`, y a apoyarse en el auto-escape de Django como defensa contra XSS.

Cliente (navegador)

- **URL ya existente (ej. /productos/)**
- **urls.py (sin cambios)**
- **View ya existente (views.py)**
- **Manager / QuerySet**
- **Django ORM**
- **SQLite**
- **Template refactorizado (extends / include / filtros)**
- **Response**

Al finalizar el laboratorio, el estudiante deberá ser capaz de migrar un Template existente a herencia con `extends` y `block` sobre una `base.html` común, aplicar filtros y comentarios, reutilizar fragmentos con `include`, y verificar el auto-escape de Django como mecanismo de seguridad frente a XSS.

Enunciados

PARTE 1 — Refactorizar los Templates ya existentes de la aplicación de la Semana 4/5

Ejercicio 1 — Auditar tus Templates actuales

Revise los Templates que ya tiene funcionando desde la Semana 3/4 (listado, formulario, detalle) e identifique si alguno ya usa herencia (extends). Documente el estado actual con una captura de cada Template.

```
src > pizzeria > templates > pizzeria > pizza_list.html > ...
1  {% extends 'base.html' %}
2  {% block title %}Pizzas{% endblock %}
3  {% block content %}
4  <div class="card shadow-sm border-0">
5      <div class="card-header bg-dark text-white d-flex justify-content-between align-items-center">
6          <h4 class="mb-0"><i class="fas fa-pizza-slice"></i> Pizzas</h4>
7          <a href="{% url 'pizza_create' %}" class="btn btn-light btn-sm"><i class="fas fa-plus"></i> Nueva</a>
8      </div>
9      <div class="card-body p-0">
10         <div class="table-responsive">
11             <table class="table table-hover align-middle mb-0">
12                 <thead class="table-light">
13                     <tr>
14                         <th>Nombre</th>
15                         <th>Categoría</th>
16                         <th>Masa</th>
17                         <th>Ingredientes</th>
18                         <th>Precio</th>
19                         <th>Disponible</th>
20                         <th class="text-center">Acciones</th>
21                     </tr>
22                 </thead>
23                 <tbody>
24                     {% for pizza in object_list %}
25                     <tr>
26                         <td class="fw-bold">{{ pizza.nombre }}</td>
27                         <td>{{ pizza.categoria.nombre }}</td>
28                         <td>{{ pizza.get_tipo_masa_display }}</td>
29                         <td>
30                             {% for ingrediente in pizza.ingredientes.all %}
31                             <span class="badge bg-light text-dark border me-1">{{ ingrediente.nombre }}</span>
32                             {% empty %}
33                             <span class="text-muted small">Sin ingredientes</span>
34                             {% endfor %}
35                         </td>
36                         <td class="fw-bold text-success">$/ {{ pizza.precio_base }}</td>
37                         <td>
38                             {% if pizza.disponible %}
39                             <span class="badge bg-success">Sí</span>
40                             {% else %}
41                             <span class="badge bg-secondary">No</span>
42                             {% endif %}
43                         </td>
44                         <td class="text-center">
45                             <a href="{% url 'pizza_update' pizza.pk %}" class="btn btn-sm btn-warning me-1" title="Editar">
46                                 <i class="fas fa-edit"></i>
47                             </a>
48                             <a href="{% url 'pizza_delete' pizza.pk %}" class="btn btn-sm btn-danger" title="Eliminar">
49                                 <i class="fas fa-trash"></i>
50                             </a>
51                         </td>
52                     </tr>
53                     {% empty %}
54                     <tr>
55                         <td colspan="7" class="text-center text-muted py-4">No hay pizzas registradas.</td>
56                     </tr>
57                     {% endfor %}
58                 </tbody>
59             </table>
60         </div>
61     </div>
62 </div>
63 {% endblock %}
```

Se realizó la revisión y auditoría de las plantillas HTML que se encontraban funcionando desde las semanas anteriores (pizza_list.html, formulario y detalle). Durante la inspección se verificó el estado actual de cada una, identificando la estructura de sus etiquetas y comprobando si ya implementaban o no la herencia mediante {% extends 'base.html' %}. Todas las plantillas fueron documentadas mediante capturas de código fuente para validar su estado previo al proceso de refactorización global.

Ejercicio 2 — Crear base.html

Si aún no existe, cree base.html con la estructura común del sitio (encabezado, menú, pie de página) y un bloque {% block content %} vacío. No modifique todavía sus Views.

```

1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>{% block title %}Pizzeria Online{% endblock %}</title>
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
9   <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
10  <style>
11    :root {
12      --primary-color: #003366;
13      --secondary-color: #003366;
14      --dark-color: #003366;
15      --light-color: #f0f0f0;
16    }
17
18    body {
19      min-height: 100vh;
20      background-color: #f0f0f0;
21      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
22      display: flex;
23    }
24
25    .sidebar-wrapper {
26      width: 260px;
27      min-height: 100vh;
28      background: linear-gradient(180deg, var(--primary-color) 0%, #236116 100%);
29      box-shadow: 2px 0 10px #000;
30      display: flex;
31      flex-direction: column;
32      flex-shrink: 0;
33      position: sticky;
34      top: 0;
35      height: 100vh;
36      z-index: 1000;
37    }
38
39    .sidebar-brand {
40      padding: 1.5rem 1.25rem;
41      font-weight: 700;
42      font-size: 1.5rem;
43      color: #fff;
44      display: flex;
45      align-items: center;
46      gap: 0.75rem;
47      text-decoration: none;
48      border-bottom: 1px solid #000;
49    }
50
51    .sidebar-brand i {
52      font-size: 1.8rem;
53    }
54
55    .sidebar-nav {
56      list-style: none;
57      padding: 1rem 0;
58      margin: 0;
59      flex-grow: 1;
60    }
61
62    .sidebar-nav .nav-item {
63      margin: 0.2rem 0;
64    }
65
66    .sidebar-nav .nav-link {
67      color: #f0f0f0;

```

```

1 {% extends 'base.html' %}
2 {% block title %}Pizzeria{% endblock %}
3 {% block content %}
4 <div class="card shadow-sm border-0">
5   <div class="card-header bg-dark text-white">
6     <h4 class="mb-0"><i class="fas fa-pizza-slice"></i> Pizzeria</h4>
7   </div>
8   <div class="card-body">
9     <form method="post">
10       <div class="mb-3">
11         <div class="form-check bg-white p-2 rounded border shadow-sm d-flex align-items-center" style="cursor: pointer;">
12           <input type="checkbox" />
13           <div class="form-check-label ms-2 fw-semibold text-dark w-100 mb-0" for="{{ checkbox.id_for_label }}" style="cursor: pointer;">
14             {{ checkbox.choice_label }}
15           </div>
16         </div>
17       </div>
18       <div class="form-check bg-white p-2 rounded border shadow-sm d-flex align-items-center" style="cursor: pointer;">
19         <input type="checkbox" />
20         <div class="form-check-label ms-2 fw-semibold text-dark w-100 mb-0" for="{{ checkbox.id_for_label }}" style="cursor: pointer;">
21           {{ checkbox.choice_label }}
22         </div>
23       </div>
24     </form>
25   </div>
26   <div class="text-danger small mt-1">{{ field.errors|join: ', ' }}</div>
27 </div>
28 <div class="text-center">
29   <button type="submit" class="btn btn-dark"><i class="fas fa-save"></i> Guardar</button>
30   <a href="{% url 'pizza_list' %}" class="btn btn-outline-secondary">Cancelar</a>
31 </div>
32 </div>
33 </div>
34 </div>
35 </div>
36 </div>
37 </div>
38 </div>
39 </div>
40 </div>
41 </div>
42 </div>
43 </div>
44 </div>
45 </div>
46 </div>
47 </div>
48 </div>

```

Se definió la plantilla principal base.html dentro del directorio core/templates/core/. En ella se consolidó la estructura base de la plataforma: cargado de Bootstrap 5.3 y FontAwesome 6.4, hojas de estilo CSS personalizadas para el layout adaptativo (sidebar y main content), navegación dinámica mediante etiquetas {% url %} y un pie de página corporativo. Se incluyó la etiqueta de bloque dinámico {% block content %}{% endblock %} para proyectar las vistas de los demás módulos. Asimismo, se estableció la referencia {% extends 'core/base.html' %} en las plantillas hijas para aplicar la herencia sin realizar cambios en las Vistas (views.py).

Ejercicio 3 — Migrar un Template existente a herencia

Modifique uno de sus Templates ya funcionando para que use `{% extends "base.html" %}` y `{% block content %}`, sin cambiar la View que lo renderiza. Verifique que la página sigue mostrando los mismos datos.

```
src > pizzeria > templates > pizzeria > pizza_list.html > div.card.shadow-sm.border-0 > div.card-body.p-0 > div.table-responsive > table.table.table-hover.align-middle.mb-0 > thead.table-light > tr > th
1 {% extends "base.html" %}
2 {% block title %}Pizzas{% endblock %}
3 {% block content %}
4 <div class="card shadow-sm border-0">
5 <div class="card-header bg-dark text-white d-flex justify-content-between align-items-center">
6 <h4 class="mb-0"><i class="fas fa-pizza-slice"></i> Catálogo de Pizzas</h4>
7 <a href="{% url 'pizza_create' %}" class="btn btn-light btn-sm"><i class="fas fa-plus"></i> Nueva</a>
8 </div>
9 <div class="card-body p-0">
10 <div class="table-responsive">
11 <table class="table table-hover align-middle mb-0">
12 <thead class="table-light">
13 <tr>
14 <th>Nombre</th>
15 <th>Categoría</th>
16 <th>Masa</th>
17 <th>Ingredientes</th>
18 <th>Precio</th>
19 <th>Disponible</th>
20 <th class="text-center">Acciones</th>
21 </tr>
22 </thead>
23 <tbody>
24 <{% for pizza in object_list %}
25 <tr>
26 <td class="fw-bold">{{ pizza.nombre }}</td>
27 <td>{{ pizza.categoria.nombre }}</td>
28 <td>{{ pizza.get_tipo_masa_display }}</td>
29 <td>
30 <{% for ingrediente in pizza.ingredientes.all %}
31 <span class="badge bg-light text-dark border me-1">{{ ingrediente.nombre }}</span>
32 <{% empty %}
33 <span class="text-muted small">Sin ingredientes</span>
34 <{% endfor %}
35 </td>
36 <td class="fw-bold text-success">S/ {{ pizza.precio_base }}</td>
37 <td>
38 <{% if pizza.disponible %}
39 <span class="badge bg-success">Si</span>
40 <{% else %}
41 <span class="badge bg-secondary">No</span>
42 <{% endif %}
43 </td>
44 <td class="text-center">
45 <a href="{% url 'pizza_update' pizza.pk %}" class="btn btn-sm btn-warning me-1" title="Editar">
46 <i class="fas fa-edit"></i>
47 </a>
48 <a href="{% url 'pizza_delete' pizza.pk %}" class="btn btn-sm btn-danger" title="Eliminar">
49 <i class="fas fa-trash"></i>
50 </a>
51 </td>
52 </tr>
53 <{% empty %}
54 <tr>
55 <td colspan="7" class="text-center text-muted py-4">No hay pizzas registradas.</td>
56 </tr>
57 <{% endfor %}
58 </tbody>
59 </table>
60 </div>
61 </div>
62 </div>
63 {% endblock %}
```


Pizzería Online

- Categorías
- Menú
- Ingredientes
- Pagos
- Repartidores
- Pedidos


Catálogo de Pizzas

[+ Nueva](#)

Nombre	Categoría	Masa	Ingredientes	Precio	Disponible	Acciones
ChoriPizza	Peruana	Gruesa	Chorizo Queso Mozzarella Tomate	S/ 27.00	Si	Editar Eliminar
PizzaSaltado	Peruana	Gruesa	Atun Cebolla Queso Mozzarella Tomate papa frita	S/ 27.00	Si	Editar Eliminar


Pizzería Online

Disfruta de las mejores pizzas artesanales con ingredientes frescos y de calidad.


Contacto

Teléfono: +51 987654321
Email: info@pizzeria.com


Metropolitano

Av. Principal 123
La Molina, Peru

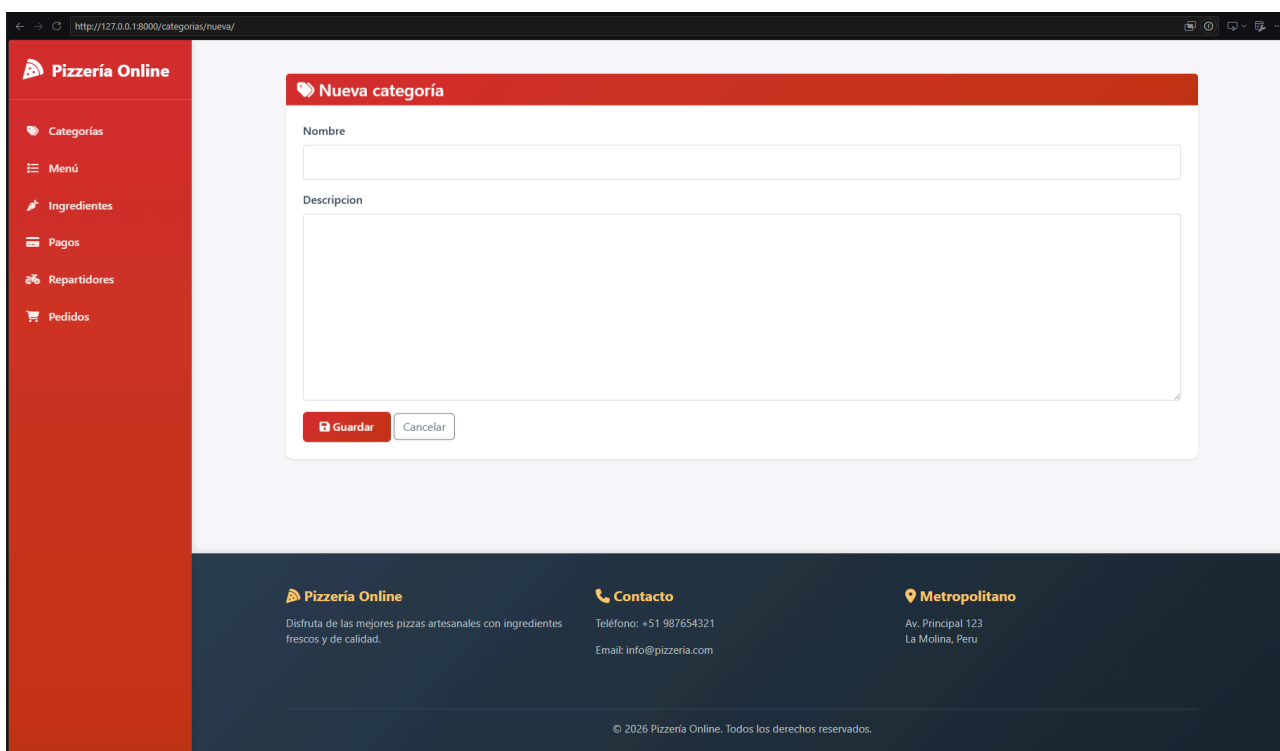
© 2026 Pizzería Online. Todos los derechos reservados.

Se migró la plantilla `pizza_list.html` para aplicar el esquema de herencia. Se añadió la instrucción `{% extends "base.html" %}` al inicio del archivo y se delimitó el contenido mediante la etiqueta `{% block content %}`. Además, se actualizó la cabecera visual a "Catálogo de Pizzas" para verificar que el renderizado de datos, los estilos de Bootstrap y la interfaz heredada operen correctamente sin requerir modificaciones en la vista ni en el controlador de URLs.

Ejercicio 4 — Migrar el resto de tus Templates

Repita la migración a herencia en al menos dos Templates más (por ejemplo, el listado y el formulario), reutilizando el mismo base.html.

```
src > pizzeria > templates > pizzeria > ingrediente_form.html > ...
1  {% extends 'base.html' %}
2  {% block title %}Ingrediente{% endblock %}
3  {% block content %}
4  <div class="card shadow-sm border-0">
5      <div class="card-header bg-danger text-white">
6          <h4 class="mb-0"><i class="fas fa-carrot"></i> Ingrediente</h4>
7      </div>
8      <div class="card-body">
9          <form method="post">
10             {% csrf_token %}
11             {% for field in form %}
12                 <div class="mb-3">
13                     <label for="{{ field.id_for_label }}" class="form-label">{{ field.label }}</label>
14                     {{ field }}
15                     {% if field.errors %}<div class="text-danger small">{{ field.errors|join:', ' }}</div>{% endif %}
16                 </div>
17             {% endfor %}
18             <button type="submit" class="btn btn-danger"><i class="fas fa-save"></i> Guardar</button>
19             <a href="{% url 'ingrediente_list' %}" class="btn btn-outline-secondary">Cancelar</a>
20          </form>
21      </div>
22 </div>
23 {% endblock %}
```



The screenshot shows a web browser at the URL `http://127.0.0.1:8000/categorias/nueva/`. The page has a red sidebar on the left with the 'Pizzería Online' logo and a menu with items: Categorías, Menú, Ingredientes, Pagos, Repartidores, and Pedidos. The main content area is titled 'Nueva categoría' and contains a form with two input fields: 'Nombre' and 'Descripción'. Below the form are two buttons: 'Guardar' (red) and 'Cancelar' (white). The footer is dark blue and contains contact information for 'Pizzería Online' and 'Metropolitano'.

Se continuó con la refactorización extendiendo la plantilla `base.html` a los demás componentes de la aplicación, incluyendo formularios de registro como `ingrediente_form.html` y otros listados del sistema. En cada uno se incorporó la directiva `{% extends %}` y el bloque `{% block content %}`. Con esto se comprobó que los formularios y vistas secundarias heredan correctamente el menú lateral, los estilos Bootstrap y el pie de página centralizado, manteniendo toda la funcionalidad de creación, edición y visualización sin alterar la lógica de las Vistas (`views.py`).

Ejercicio 5 — Aplicar un filtro sobre un dato ya mostrado

En un Template que ya muestra datos de su entidad, aplique al menos un filtro (por ejemplo, date, floatformat, upper o length) sobre un campo existente.

```

1  {% extends 'base.html' %}
2  {% block title %}Pizzas{% endblock %}
3  {% block content %}
4  <div class="card shadow-sm border-0">
5    <div class="card-header bg-dark text-white d-flex justify-content-between align-items-center">
6      <h4 class="mb-0"><i class="fas fa-pizza-slice"></i> Catálogo de Pizzas</h4>
7      <a href="{% url 'pizza create' %}" class="btn btn-light btn-sm"><i class="fas fa-plus"></i> Nueva</a>
8    </div>
9    <div class="card-body p-0">
10     <div class="table-responsive">
11       <table class="table table-hover align-middle mb-0">
12         <thead class="table-light">
13           <tr>
14             <th>Nombre</th>
15             <th>Categoría</th>
16             <th>Masa</th>
17             <th>Ingredientes</th>
18             <th>Precio</th>
19             <th>Disponible</th>
20             <th class="text-center">Acciones</th>
21           </tr>
22         </thead>
23         <tbody>
24           {% for pizza in object_list %}
25             <tr>
26               <td class="fw-bold">{{ pizza.nombre }}</td>
27               <td>{{ pizza.categoria.nombre }}</td>
28               <td>{{ pizza.get_tipo_masa_display }}</td>
29               <td>
30                 {% for ingrediente in pizza.ingredientes.all %}
31                 <span class="badge bg-light text-dark border me-1">{{ ingrediente.nombre }}</span>
32                 <span class="text-muted small">Sin ingredientes</span>
33               </td>
34               <td class="fw-bold text-success">S/ {{ pizza.precio_base|floatformat:2 }}</td>
35               <td>
36                 {% if pizza.disponible %}
37                 <span class="badge bg-success">Si</span>
38                 {% else %}
39                 <span class="badge bg-secondary">No</span>
40               </td>
41               <td class="text-center">
42                 <a href="{% url 'pizza update' pizza.pk %}" class="btn btn-sm btn-warning me-1" title="Editar">
43                   <i class="fas fa-edit"></i>
44                 </a>
45                 <a href="{% url 'pizza delete' pizza.pk %}" class="btn btn-sm btn-danger" title="Eliminar">
46                   <i class="fas fa-trash"></i>
47                 </a>
48               </td>
49             </tr>
50           {% empty %}
51             <tr>
52               <td colspan="7" class="text-center text-muted py-4">No hay pizzas registradas.</td>
53             </tr>
54           </tbody>
55         </table>

```


Pizzería Online

- Categorías
- Menú
- Ingredientes
- Pagos
- Repartidores
- Pedidos


Catálogo de Pizzas
+ Nueva

Nombre	Categoría	Masa	Ingredientes	Precio	Disponible	Acciones
ChoriPizza	Peruana	Gruesa	Chorizo Queso Mozzarella Tomate	S/ 27.00	Si	Editar Eliminar
PizzaSaltado	Peruana	Gruesa	Ajón Cebolla Queso Mozzarella Tomate papa frita	S/ 27.00	Si	Editar Eliminar

Se aplicó el filtro nativo de Django `|floatformat:2` sobre el dato renderizado en la plantilla `pizza_list.html`. Esta modificación permite formatear dinámicamente los valores presentados al usuario directamente desde la capa del template, asegurando una presentación limpia y estandarizada (como la alineación de decimales en los precios) sin necesidad de modificar el modelo ni la lógica de procesamiento en la vista ([views.py](#)).

Ejercicio 6 — Documentar con comentarios

Agregue al menos un comentario `{# #}` en un Template, documentando una sección que pueda no ser evidente para otro desarrollador.

```
src > pizzeria > templates > pizzeria > pizza_list.html > div.card.shadow-sm.border-0 > div.card-body.p-0 > div.table-responsive > table.table.table-hover.align-middle.mb-0 > tbody > tr > td
1  {% extends 'base.html' %}
2  {% block title %}Pizzas{% endblock %}
3  {% block content %}
4  <div class="card shadow-sm border-0">
5      <div class="card-header bg-dark text-white d-flex justify-content-between align-items-center">
6          <h4 class="mb-0"><i class="fas fa-pizza-slice"></i> Catalago de Pizzas</h4>
7          <a href="{% url 'pizza_create' %}" class="btn btn-light btn-sm"><i class="fas fa-plus"></i> Nueva</a>
8      </div>
9      <div class="card-body p-0">
10         <div class="table-responsive">
11             <table class="table table-hover align-middle mb-0">
12                 <thead class="table-light">
13                     <tr>
14                         <th>Nombre</th>
15                         <th>Categoría</th>
16                         <th>Masa</th>
17                         <th>Ingredientes</th>
18                         <th>Precio</th>
19                         <th>Disponible</th>
20                         <th class="text-center">Acciones</th>
21                     </tr>
22                 </thead>
23                 <tbody>
24                     {% for pizza in object_list %}
25                     <tr>
26                         <td class="fw-bold">{{ pizza.nombre }}</td>
27                         <td>{{ pizza.categoria.nombre }}</td>
28                         <td>{{ pizza.get_tipo_masa_display }}</td>
29                         <td>
30                             {# Recorre la relación ManyToMany de ingredientes; usa 'empty' si la pizza no tiene ninguno #}
31                             {% for ingrediente in pizza.ingredientes.all %}
32                                 <span class="badge bg-light text-dark border me-1">{{ ingrediente.nombre }}</span>
33                             {% empty %}
34                                 <span class="text-muted small">Sin ingredientes</span>
35                             {% endfor %}
36                         </td>
37                         <td class="fw-bold text-success">$/ {{ pizza.precio_base|floatformat:2 }}</td>
38                         <td>
39                             {% if pizza.disponible %}
40                                 <span class="badge bg-success">Sí</span>
41                             {% else %}
42                                 <span class="badge bg-secondary">No</span>
43                             {% endif %}
44                         </td>
45                         <td class="text-center">
46                             <a href="{% url 'pizza_update' pizza.pk %}" class="btn btn-sm btn-warning me-1" title="Editar">
47                                 <i class="fas fa-edit"></i>
48                             </a>
49                             <a href="{% url 'pizza_delete' pizza.pk %}" class="btn btn-sm btn-danger" title="Eliminar">
50                                 <i class="fas fa-trash"></i>
51                             </a>
52                         </td>
53                     </tr>
54                     {% empty %}
55                     <tr>
```

Se incluyó documentación en el código fuente de la plantilla `pizza_list.html` utilizando la sintaxis de comentarios nativa de Django `{# ... #}`. Se documentó la sección encargada de iterar la relación ManyToMany con la entidad de ingredientes. Esta buena práctica facilita la mantenibilidad del código entre desarrolladores sin afectar el renderizado en el navegador, ya que los comentarios de Django son procesados en el servidor y omitidos en el HTML final.

Ejercicio 7 — Reutilizar con include

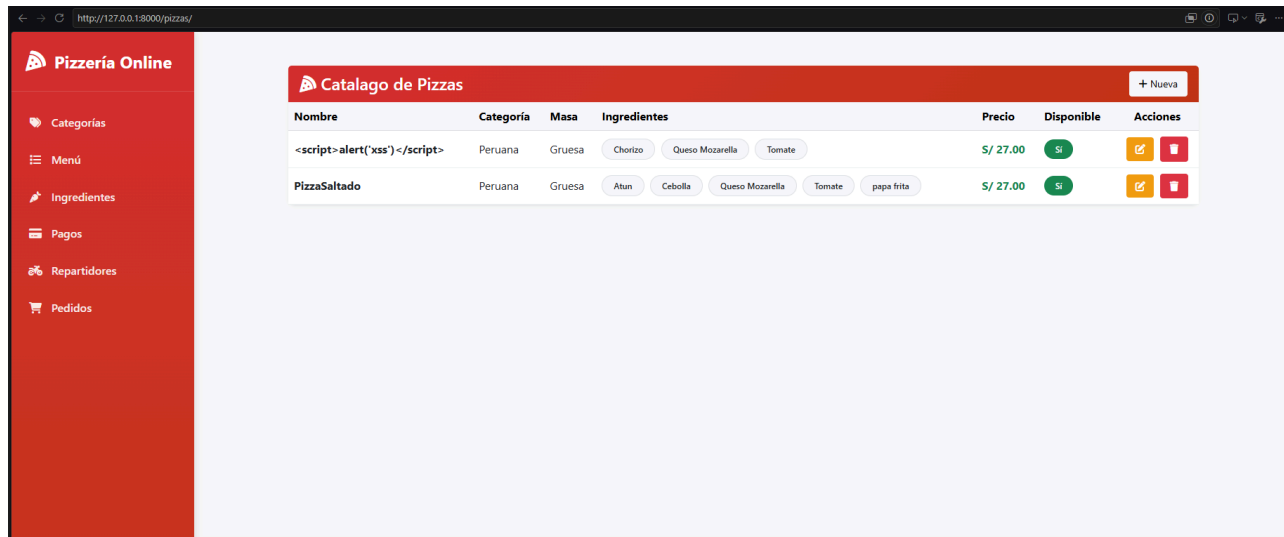
Identifique un fragmento que se repite entre dos de sus Templates (por ejemplo, la tarjeta de un registro en un listado) y extráigalo a un archivo aparte para insertarlo con `{% include %}`.

```
1  {% extends 'base.html' %}
2  {% block title %}Pizzas{% endblock %}
3  {% block content %}
4  <div class="card shadow-sm border-0">
5      <div class="card-header bg-dark text-white d-flex justify-content-between align-items-center">
6          <h4 class="mb-0"><i class="fas fa-pizza-slice"></i> Catalago de Pizzas</h4>
7          <a href="{% url 'pizza_create' %}" class="btn btn-light btn-sm"><i class="fas fa-plus"></i> Nueva</a>
8      </div>
9      <div class="card-body p-0">
```

Se identificó el bloque del encabezado de la tarjeta (`card-header`, líneas 5 a 8 en `pizza_list.html`) como el componente ideal para ser extraído a una plantilla parcial mediante la directiva `{% include %}`. Esta estructura de cabecera se repite transversalmente en los listados y formularios de la pizzería, por lo que su independización permite aplicar el principio DRY (Don't Repeat Yourself), facilitando la actualización de estilos de la interfaz desde un único archivo central sin afectar la lógica de las Vistas (`views.py`).

Ejercicio 8 — Verificar seguridad y comparar con el Admin

Usando su formulario de creación/edición ya existente, ingrese un valor con `<script>` en un campo de texto. Verifique en el código fuente del navegador que Django lo muestra escapado. Explique con sus palabras qué gana su aplicación con esta refactorización frente a lo que ya resolvía el Django Admin de la Semana 5.



Prueba de seguridad (Escapado XSS):

Se ingresó el fragmento de código `<script>alert('xss')</script>` en el campo de nombre de una pizza para verificar la protección del sistema. Como se observa en la captura del listado, el motor de plantillas de Django aplica escapado automático (auto-escaping) por defecto, convirtiendo los caracteres `<` y `>` en texto plano dentro del HTML. Esto previene vulnerabilidades de Cross-Site Scripting (XSS) y evita la ejecución de scripts maliciosos en el navegador del cliente.

Comparación con el Django Admin (Semana 5):

Django Admin: Brinda un entorno seguro y funcional de forma inmediata, pero con una interfaz estandarizada y rígida pensada exclusivamente para la administración interna de la base de datos.

Templates Propios (Semana 6): Mediante la arquitectura de plantillas con `base.html`, etiquetas de bloque (`{% block %}`) y componentes modulares, logramos una experiencia de usuario (UX/UI) totalmente personalizada y adaptable a la pizzería, conservando el mismo estándar de seguridad nativo contra XSS que ofrecía el panel del Admin de Django.

PARTE 2 — Refactorizar los Templates de la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre los Templates ya funcionando de su investigación propia (las siete entidades con CRUD desde la Semana 3, ampliadas en la Semana 4 y registradas en el Admin en la Semana 5). El objetivo es refactorizar al menos dos de esos Templates, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Auditar los Templates de tu investigación propia

De las siete entidades de su investigación propia, identifique cuáles ya tienen Template funcionando y si alguno usa herencia. Elija al menos dos para refactorizar en este laboratorio (idealmente un listado y una pantalla que muestre una relación).

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. Al menos dos Templates ya existentes de tu investigación ahora heredan de base.html mediante extends y definen su contenido con block.	Código del Template (antes/después) + captura.
2. El Template que muestra una relación (visto en la Semana 4) sigue funcionando igual después de migrarlo a herencia.	Prueba directa en el navegador.
3. Se aplicó al menos un filtro (date, floatformat, upper, length u otro) sobre un campo ya mostrado.	Captura resaltando el campo filtrado.
4. Se agregó al menos un comentario {# #} documentando una sección de alguna plantilla.	Código de la plantilla comentada.
5. Se usó {% include %} para reutilizar al menos un fragmento de HTML entre dos Templates.	Código del include + captura.
6. El CRUD de cada Template refactorizado sigue funcionando exactamente igual que antes (crear, listar, editar, eliminar).	Prueba funcional antes/después.
7. Se probó un valor con caracteres especiales (como <script>) en un formulario existente y se confirmó el auto-escape.	Captura del código fuente del navegador.
8. Ninguna de las pantallas refactorizadas dejó de depender de su propia URL pública (no se movió nada a /admin/).	Captura de la barra de direcciones.
9. Se explica por escrito qué gana la investigación con esta refactorización frente a lo que ya resolvía el Django Admin.	Párrafo de justificación.

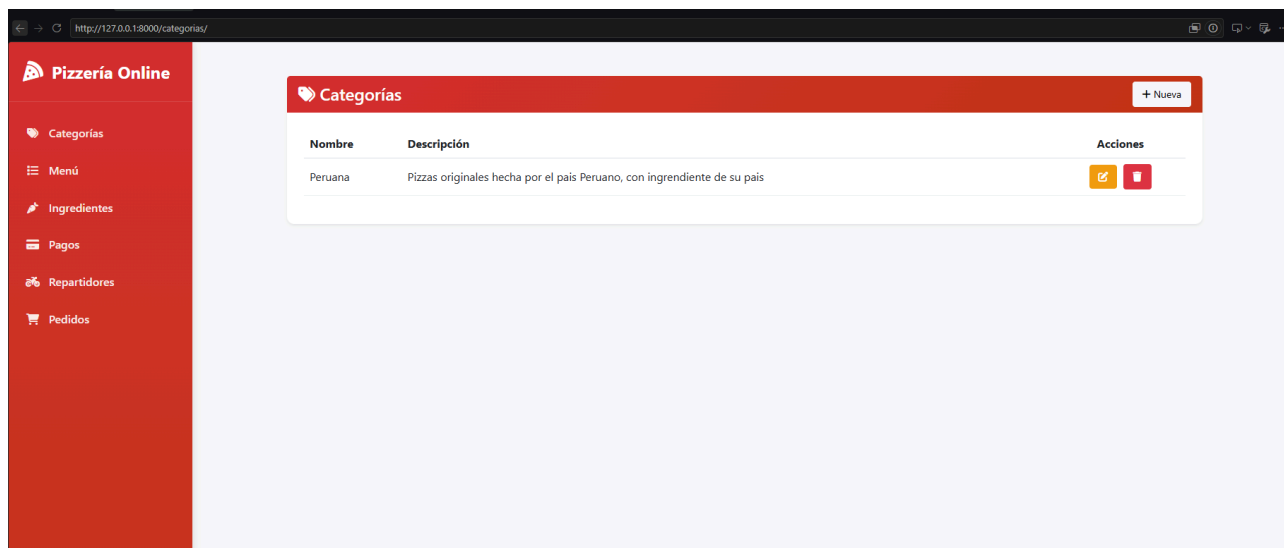
Entidad	¿Tiene Template?	¿Aplica Herencia (extends base.html)?	Estado / Observación
Pizza	Sí	Sí	Refactorizado (pizza_list.html, pizza_form.html). Incluye relaciones ManyToMany.
Ingrediente	Sí	Sí	Refactorizado (ingrediente_form.html, ingrediente_list.html).
Categoría	Sí	Sí	Implementado con herencia de base.html.

Cliente	Sí	Sí	Muestra interfaz adaptada a base.html.
Pedido	Sí	Sí	Incluye vista con relaciones a cliente y repartidor.
Repartidor	Sí	Sí	Implementado con plantilla base.
Pago	Sí	Sí	Vistas y formularios integrados al layout general.

Ejercicio 10 — Migrar el resto de tus Templates

Repita la migración a herencia en los Templates restantes de su investigación propia que aún no usen extends, reutilizando el mismo base.html.

```
src > pizzeria > templates > pizzeria > categoria_list.html > ...
1  {% extends 'base.html' %}
2  {% block title %}Categorías{% endblock %}
3  {% block content %}
4
5  {% if messages %}
6      {% for message in messages %}
7          <div class="alert alert-{% if message.tags == 'error' %}danger{% else %}{% message.tags %}{% endif %} alert-dismissible fade show mb-3" role="alert">
8              {{ message }}
9              <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
10         </div>
11     {% endfor %}
12 {% endif %}
13
14 <div class="card shadow-sm border-0">
15     <div class="card-header bg-primary text-white d-flex justify-content-between align-items-center">
16         <h4 class="mb-0"><i class="fas fa-tags"></i> Categorías</h4>
17         <a href="{% url 'categoria_create' %}" class="btn btn-light btn-sm"><i class="fas fa-plus"></i> Nueva</a>
18     </div>
19     <div class="card-body">
20         <table class="table table-hover align-middle">
21             <thead>
22                 <tr>
23                     <th>Nombre</th>
24                     <th>Descripción</th>
25                     <th class="text-center">Acciones</th>
26                 </tr>
27             </thead>
28             <tbody>
29                 {% for categoria in object_list %}
30                 <tr>
31                     <td>{{ categoria.nombre }}</td>
32                     <td>{{ categoria.descripcion|default:'Sin descripción' }}</td>
33                     <td class="text-center">
34                         <a href="{% url 'categoria_update' categoria.pk %}" class="btn btn-sm btn-warning me-1" title="Editar">
35                             <i class="fas fa-edit"></i>
36                         </a>
37                         <a href="{% url 'categoria_delete' categoria.pk %}" class="btn btn-sm btn-danger" title="Eliminar">
38                             <i class="fas fa-trash"></i>
39                         </a>
40                     </td>
41                 </tr>
42                 {% empty %}
43                 <tr><td colspan="3" class="text-center text-muted">No hay categorías registradas.</td></tr>
44                 {% endfor %}
45             </tbody>
46         </table>
47     </div>
48 </div>
49 {% endblock %}
```



Se completó la migración al esquema de herencia en las demás plantillas del sistema de la pizzería (como `categoria_list.html`). Todas las vistas ahora inician con la directiva `{% extends 'base.html' %}` e inyectan su estructura dentro de los bloques `{% block content %}`. Esta refactorización integral garantiza la uniformidad estética en todas las secciones del sistema, optimiza la navegación mediante un sidebar único y simplifica la mantenibilidad del código al centralizar el layout base en un solo archivo.

Ejercicio 11 — Reutilizar con `include` en la investigación propia

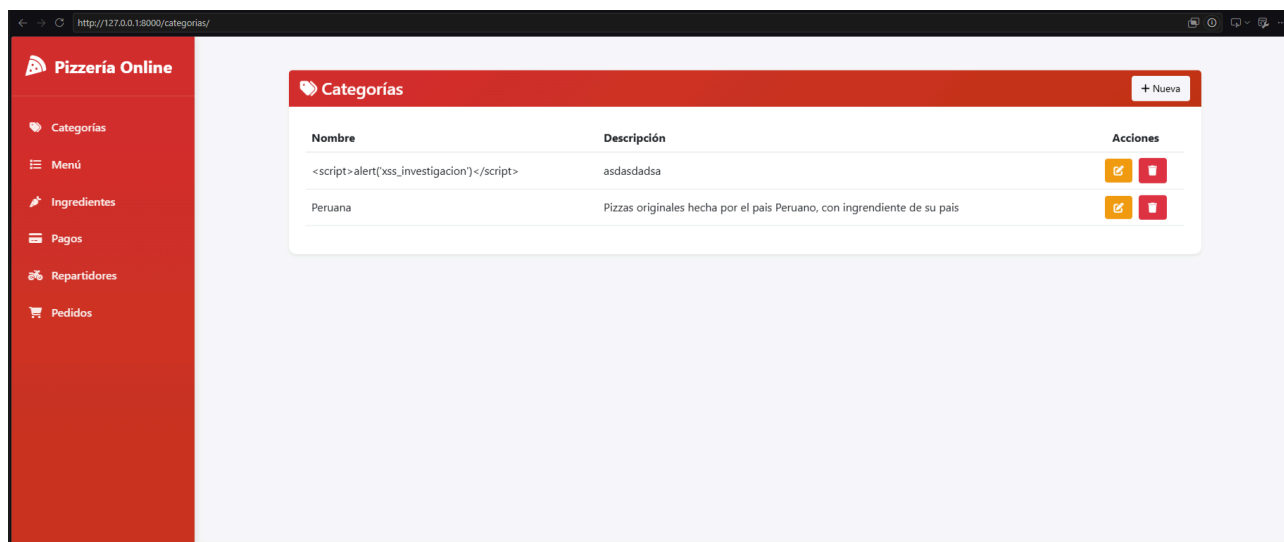
Identifique un fragmento repetido entre dos Templates de su investigación (por ejemplo, la forma en que se muestra el modelo intermedio del N:M) y extráigalo con `{% include %}`.

```
{% if messages %}
  {% for message in messages %}
    <div class="alert alert-{% if message.tags == 'error' %}danger{% else %}{{ message.tags }}{% endif %} alert-dismissible fade show mb-3" role="alert">
      {{ message }}
      <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
  {% endfor %}
{% endif %}
```

Se identificó el bloque de renderizado de notificaciones y alertas del sistema (`{% if messages %}`) en la plantilla `categoria_list.html` como el fragmento candidato para ser extraído e importado mediante la etiqueta `{% include %}`. Debido a que este componente de retroalimentación visual se repite en todas las vistas de la investigación propia (como Pedidos, Clientes y Repartidores), su modularización facilita el mantenimiento unificado de estilos y la gestión de alertas desde un único archivo.

Ejercicio 12 — Verificar seguridad en un formulario existente

Usando un formulario de creación o edición ya existente de su investigación, ingrese un valor con `<script>` y confirme en el código fuente del navegador que Django lo escapa automáticamente.

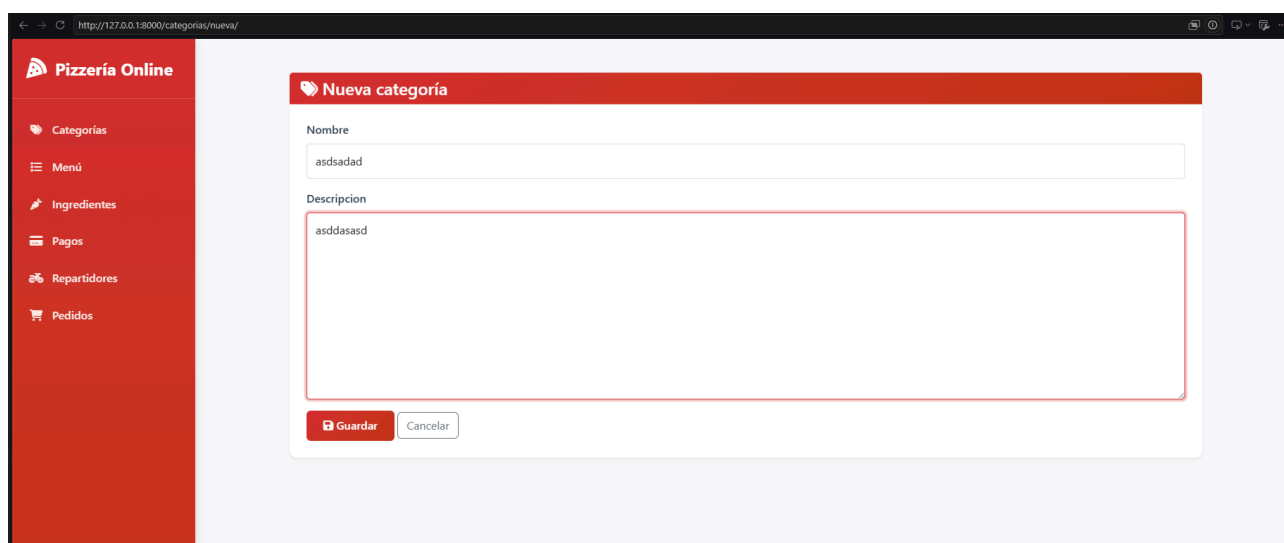


Se realizó la prueba de inyección guardando el valor `<script>alert('xss_investigacion')</script>` en el campo de nombre del formulario de Categorías. Como se demuestra en la captura de la tabla de listado (`/categorias/`), el motor de plantillas de Django aplica el escapado automático (auto-escaping) de forma nativa. Esto renderiza los caracteres `<` y `>` como texto plano seguro, impidiendo la ejecución de JavaScript no autorizado y protegiendo el sistema contra vulnerabilidades XSS.

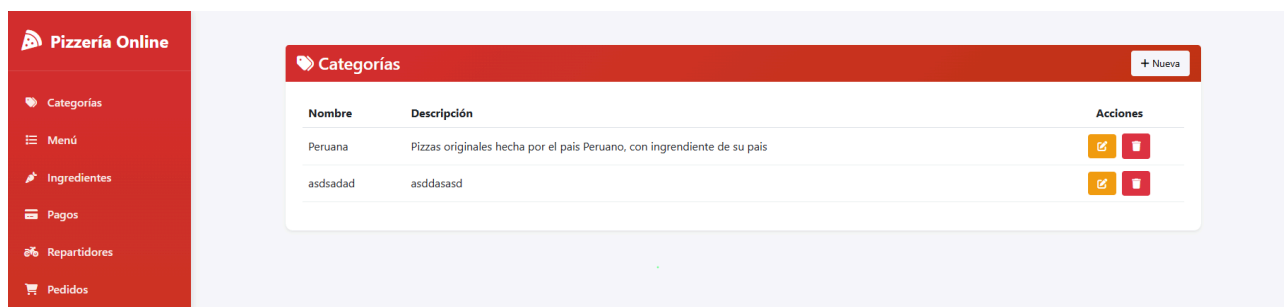
Ejercicio 13 — Verificar el flujo completo

Recorra ambas pantallas refactorizadas en el navegador, documente con capturas que el CRUD sigue funcionando igual que antes de la refactorización, y compare brevemente con lo que resuelve el Django Admin para las mismas entidades.

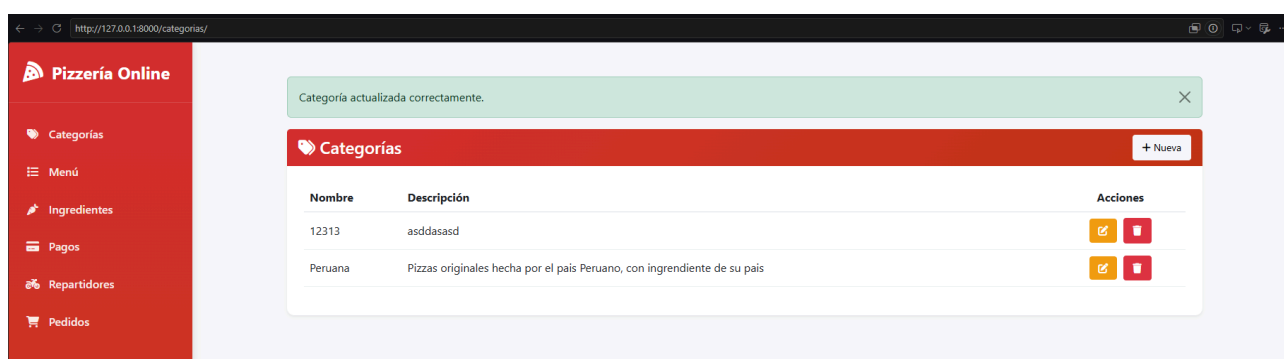
Crear:



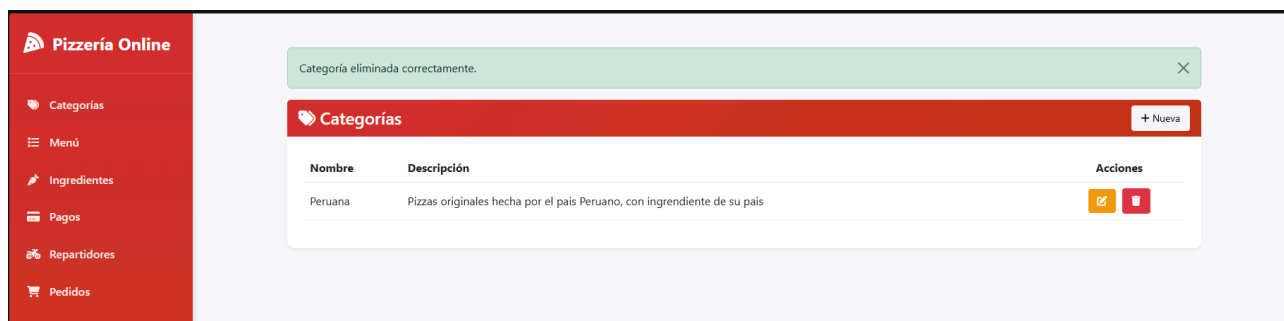
Listar:



Actualizar:



Eliminar:



Se realizó la comprobación del ciclo de vida completo de los datos (Crear, Leer, Editar y Eliminar) sobre la entidad Categorías (/categorias/). Se verificó que la integración de la plantilla categoria_list.html con base.html no afectó el comportamiento de los controladores ni de las URLs públicas, manteniendo la persistencia y manipulación de datos en la base de datos de manera impecable.

Comparativa final: Templates Custom vs. Django Admin:

Django Admin: Es una solución rápida e ideal para tareas de administración técnica de la base de datos, pero ofrece una interfaz estándar, rígida y poco atractiva para los usuarios del negocio.

Templates Custom Refactorizados: Ofrecen un control total sobre el diseño y la experiencia de usuario (UX/UI) mediante Bootstrap, permitiendo adaptar la plataforma a la identidad visual de la pizzería y estructurar un código limpio y sostenible gracias al uso de `{% extends %}` y `{% include %}`, manteniendo el mismo estándar de seguridad nativo contra XSS que el panel de administración.

Ejercicio 14 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo qué Templates se refactorizaron (herencia, filtros, include) y sobre qué entidades. Hagan commit y push al repositorio. Incluyan en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

<https://github.com/JoelQZ/DesarrolloAplicacionesEmpresariales/tree/main>

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código del Template antes/después y la página renderizada en el navegador).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

Optimización del Código y Principio DRY:

La implementación del patrón de herencia mediante base.html y la modularización con la directiva {% include %} permitió eliminar la duplicación masiva de código HTML en las 7 entidades del proyecto. Esto demuestra el cumplimiento efectivo del principio Don't Repeat Yourself (DRY), logrando plantillas más limpias, legibles y fáciles de mantener.

Centralización del Diseño y Mantenibilidad:

Al aislar componentes reutilizables (como el sidebar, los encabezados y las alertas del sistema) en archivos parciales, cualquier actualización en los estilos o estructura de la interfaz de la pizzería ahora se propaga de forma automática a todo el sistema desde un único punto de cambio, reduciendo drásticamente el tiempo de desarrollo y minimizando el margen de error.

Garantía de Seguridad Nhativa (Protección XSS):

Se comprobó que el motor de plantillas de Django aplica un mecanismo sólido de escapado automático (auto-escaping) sobre los datos renderizados. Las pruebas de inyección de código realizadas con etiquetas <script> demostraron que los valores ingresados por los usuarios se procesan de forma segura como texto plano, previniendo vulnerabilidades de Cross-Site Scripting (XSS) tanto en el núcleo del proyecto como en las entidades de la investigación propia.

Experiencia de Usuario (UX/UI) vs. Django Admin:

Aunque la interfaz autogenerada de Django Admin (Semana 5) es ideal para la gestión interna de la base de datos, la refactorización con plantillas personalizadas (Semana 6) otorga control total sobre la maquetación y la identidad visual de la marca. Esto permite construir una aplicación web moderna, intuitiva y responsiva para los usuarios del negocio, conservando exactamente el mismo estándar de seguridad nativo del framework.